



Binary Creation

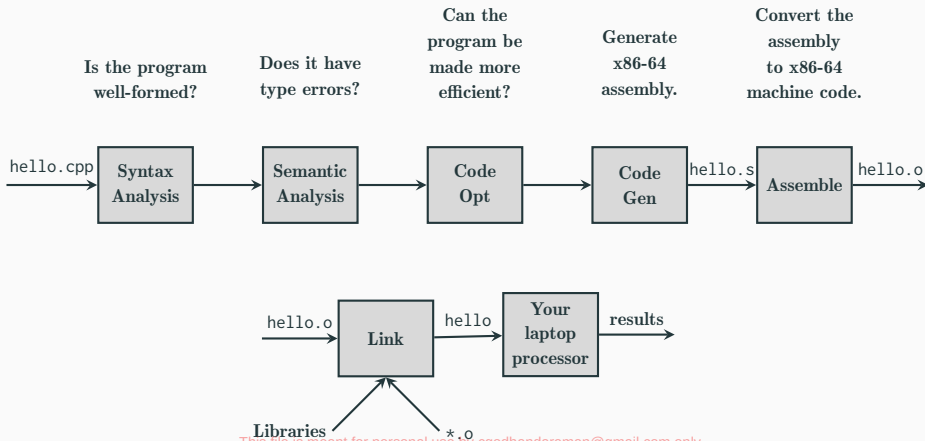


1. The Compilation Pipeline



The Compilation Pipeline

- The compilation pipeline for the C++ compiler for x86-64 (gcc)

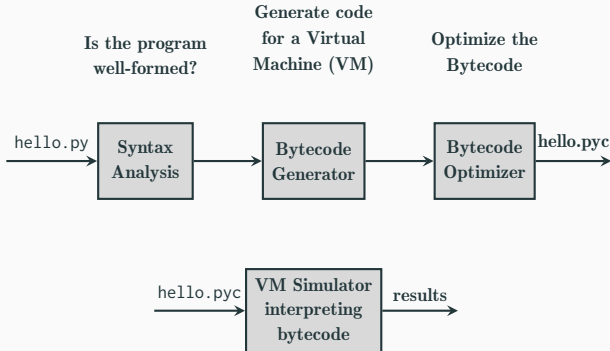


This file is meant for personal use by cgodhandaraman@gmail.com only.



The Interpretation Pipeline

- The interpretation pipeline for Python.



- Question: Where are type errors detected? Where is the program linked to libraries?**
- Tentative answer: During bytecode interpretation.**



Examining the Compilation Pipeline for GCC

Break the compilation into stages using switches:

- `g++ -E hello.cpp -o hello.ii` // Preprocessing only
- `g++ -S hello.ii -o hello.s` // Compile to assembly
- `g++ -c hello.s -o hello.o` // Assemble to object file
- `g++ hello.o -o hello` // Link to produce executable

Single-shot pipeline with intermediate files:

- `g++ -save-temps -v hello.cpp -o hello`
 - Keeps: `hello.ii`, `hello.s`, `hello.o`
 - Shows detailed internal steps (calls to `cc1plus`, `as`, `ld`)



Internal View: What GCC Really Invokes

Under the hood, g++ orchestrates several specialized tools:

- **Preprocessing and Compilation:**
 - `cc1plus` compiles `.cpp` or `.ii` to `.s`
 - Example: `/usr/lib/gcc/.../cc1plus hello.cpp -o hello.s`
- **Assembling:**
 - `as` converts `.s` to `.o`
 - Example: `as hello.s -o hello.o`
- **Linking:**
 - `collect2` is invoked as the frontend to `ld`
 - Links object files with startup code and libraries
 - Example: `collect2 ... -o hello`

Dumping after optimization passes:

- `-fdump-tree-all`, `-fdump-tree-original`, etc.
- Dumps GIMPLE IRs after various optimization passes
- Add `-fopt-info` to see optimization summaries



2. Build Tools: make and cmake



Why Build Tools?

Building an executable from interdependent sources, headers, configs, and libraries can be complex.

- Multiple source files with complex dependencies.
- Different compilers and options.
- Linkage to external libraries.
- Different build configurations (debug, release, testing).
- Setting of Environment variables and paths.

An additional efficiency concern:

- Full recompilation for minor changes is inefficient.
- Incremental builds require careful dependency tracking.



make example

A project consisting of the following compilation units:

main.c

```
1  #include <stdio.h>
2  #include <mathutils.h>
3  int main() {
4      int x;
5      printf("Enter a number: ");
6      scanf("%d", &x);
7      int result = square(x);
8      printf("The square of %d is %d\n", x,
9      result);
10     return 0;
11 }
```

mathutils.c

```
1  #include <mathutils.h>
2
3  int square(int n) {
4      return n * n;
5  }
```

mathutils.h

```
1  #ifndef MATHUTILS_H
2  #define MATHUTILS_H
3
4  int square(int n);
5
6  #endif
```



From Manual Builds to Build Tools

Manually building the project: bash

```
1 $ gcc -o main main.c mathutils.c -I.
```

- What if the executable has to be rebuilt multiple times?
- What if it depends on 100 files, and you change just two?

Solution: Use make

- Write a **Makefile** to specify dependencies and compilation commands.
- Use the **make** command for incremental builds.

Even better: Use cmake

- Write a file **CMakeLists.txt**.
- Use **cmake** to generate the **Makefile**.
- Portable, scalable, and widely supported across **IDEs** and **platforms**.



Building using make

Makefile: v1

```
1  main: main.c mathutils.c
2      gcc -o main main.c mathutils.c -I.
```

Build:

```
1  $ make
```

- Line 1: The target **main** depends on **main.c** and **mathutils.c**. If either file changes, recompilation occurs.
- Line 2: Specifies the command to build the executable **main** using **gcc**. The space before **gcc** is a single tab.



Variables in Makefile

Makefile: v2

```
1 CC = gcc
2 CPPFLAGS = -I.
3
4 main: main.o mathutils.o
5     $(CC) -o main main.o mathutils.o
6
7 #%.o: %.c
8 #     $(CC) $(CPPFLAGS) $(CFLAGS) -c -o $@ $<
```

If you want to switch to a different C compiler, say `clang`, just replace `gcc` with `clang` in the first line.

1. The executable `main` depends on the object files `main.o` and `mathutils.o`.
2. What does `main.o` depend on? `make` figures that out using the implicit rule (commented).
3. If `mathutils.c` is changed, only this file is recompiled.
4. The dependence of `main` on `mathutils.h` is not mentioned anywhere. Even if this file is changed, no recompilation is done.



Pattern Rules and Automatic Variables in make

Makefile: v3

```
1 CC = gcc
2 CPPFLAGS = -I.
3 DEPS = mathutils.h
4
5 # final target
6 main: main.o mathutils.o
7     $(CC) -o $@ $^
8
9 %.o: %.c $(DEPS)
10    $(CC) -c -o $@ $< $(CPPFLAGS)
```

Observations:

- Line 9: Each .o file depends on the .c file with the same name.
- \$@ = the left-hand side of the rule (the target).
- \$< = the first dependee on the right-hand side.
- \$^ = the full list of dependees.



Organizing Directories for Builds

Structure your project as follows:

1. Put all `.h` files in an `include/` directory
2. Place all `.c` or `.cpp` source files in a `src/` directory
3. Store local libraries (e.g., `.a` or `.so`) in a `lib/` directory
4. Hide intermediate object files (`.o`) in a dedicated `obj/` directory

This layout helps keep the project clean, maintainable, and scalable for larger builds.



Makefile: v4

```
1 CC=gcc
2 IDIR=include
3 CPPFLAGS=-I$(IDIR)
4 ODIR=obj
5 SDIR=src
6
7 _DEPS = makeutils.h
8 DEPS = $(patsubst %, $(IDIR)/%, $(_DEPS))
9 _OBJ = main.o mathutils.o
10 OBJ = $(patsubst %, $(ODIR)/%, $(_OBJ))
11 _SRC = main.c mathutils.c
12 SRC = $(patsubst %, $(SDIR)/%, $(_SRC))
13
14 main: $(OBJ)
15     $(CC) -o $@ $^ $(CPPFLAGS)
16 $(ODIR)/%.o: $(SDIR)/%.c $(DEPS)
17     $(CC) -c -o $@ $< $(CPPFLAGS)
18
19 .PHONY: clean
20 clean:
21     rm -f $(ODIR)/*.o
```



Observations

Directory structure:

```
1 .
2 |-- include
3 |   |-- mathutils.h
4 |-- Makefile4
5 |-- obj
6 |-- src
7 |-- main.c
8 |-- mathutils.c
```

- **\$(patsubst PATTERN, REPLACEMENT, TEXT):** Finds whitespace-separated words in **TEXT** that match **PATTERN**, and replaces them with **REPLACEMENT**. Here, **%** matches any number of characters within a word.
- The **makefile** includes a rule for cleaning up intermediate/object files:
\$ make clean

The **.PHONY** directive ensures **make** treats **clean** as a target, not a file.



CMake is a build-system generator that define the build process in a platform-independent way.

- Requires a configuration file **CMakeLists.txt**
- This describes:
 - Which source files to compile
 - Which dependencies to link
 - Which compilers to use?
 - Which build-system to generate.
- The syntax of **CMakeLists.txt** is simpler than **Makefiles**.



Flexibility and automation in CMake

- The default build-system on Linux is **Makefile**, but **CMake** can also generate build files for:
 - Xcode (macOS)
 - Eclipse
 - Visual Studio
- **Cross-Platform Discovery of Compilers and Libraries**
 - **CMake** detects compilers, linkers, and build tools.
 - Instead of manually specifying paths:
 - For standard system libraries (e.g., **m**, **pthread**), **CMake** simply passes the correct linker flags (e.g., **-lm**, **-lpthread**).
 - For non-standard or optional libraries (e.g., **OpenSSL**, **Boost**), **CMake** locates headers and compiled libraries and generate appropriate compiler and linker flags(e.g., **-I**, **-L**).



Example: Building the earlier project with CMake

Directory Structure:

```
1  .
2  |-- build/
3  |-- CMakeLists.txt
4  |-- include/
5  |   |-- mathutils.h
6  |-- src/
7  |   |-- main.c
8  |   |-- mathutils.c
```

CMakeLists.txt

```
1  cmake_minimum_required(VERSION 3.22.1)
2  project(main_test)
3  include_directories(include)
4  file(GLOB SOURCES "src/*.c")
5  add_executable(main ${SOURCES})
```

Note: To avoid clutter, the `build/` directory is created to store the generated files.



CMake: Automatic Identifications

When you run `cmake .`, it performs several automatic checks:

- Detects the C compiler (e.g., `gcc 11.4.0`)
- Detects the C++ compiler (e.g., `g++ 11.4.0`)
- Verifies if the C compiler works (compiles and links a test program)
- Verifies if the C++ compiler works
- Detects ABI information for both C and C++ compilers
- Detects supported compile-time features (e.g., `C++11`, `C99`)
- Completes configuration and generates build files (e.g., `Makefile`)
- Indicates the path where build files are written



A Brief Detour: Creating a (shared) library

What is a shared library?

- Compiled code ready for use.
- Example: **libm** is the standard C math library.
- Two aspects to a shared library:
 - The header file **math.h** in **/usr/include** contains function prototypes for type-checking.
 - The shared object file **libm.so** (in **/usr/lib/x86_64-linux-gnu**) contains the code.

Using **CMake**, shall convert **mathutils.c** into a shared library.



Files:

Creating a Shared Library

The header `mathutils.h` in `include/`:

```
1  #ifndef MATHUTILS_H
2  #define MATHUTILS_H
3  int square(int n);
4  #endif;
```

This is our great library that we want the world to use:

`mathutils.c`

```
1  #include <mathutils.h>
2
3  int square(int n) {
4      return n * n;
5  }
```

And a test function `main.c` to use the library:

```
1  #include <stdio.h>
2  #include <mathutils.h>
3
4  int main() {
5      int x;
6      printf("Enter a number: ");
7      scanf("%d", &x);
8      int result = square(x);
9      printf("The square of %d is %d\n", x,
10         result);
11     return 0;
12 }
```



Building the library mathutils manually

Build the library `libmathutils.so` manually:

```
1 $ gcc -c -fpic -Iinclude mathutils.c
2 $ gcc -shared -o libmathutils.so mathutils.o
```

- `-fpic` generates position-independant code for dynamic linking. More about this later.
- Library executables must have names like `lib*.so`

Compile and run the program:

```
1 $ export LD_LIBRARY_PATH=$(pwd):$LD_LIBRARY_PATH # For the dynamic linker (ld-linux.so) at runtime
2 $ gcc -L$(pwd) -o main main.c -lmathutils -Iinclude # For the static linker (ld) during build time
3 # $ gcc -L$(pwd) -o main main.c -lmathutils -Iinclude -Wl,-rpath,$(pwd) Alternative to setting LD_LIBRARY_PATH
4 $ ./main
5 Enter a number: 9
6 The square of 9 is 81
```

`lmathutils` links against `libmathutils.so`, and `LD LIBRARY PATH` tells the system where to find it at runtime.



Using cmake to Create a Shared Library

We shall illustrate two things:

1. How to create a library and then use it to create an executable.
2. How to create a hierarchy of CMakeLists.txt files.

Project Layout:

```
1  .
2  |-- build/           # to build
3  |-- CMakeLists.txt   # cmake script to build the executable
4  |-- include/
5  |   |-- mathutils.h  # header for the library
6  |-- shared/
7  |   |-- mathutils.c   # source for the library
8  |   |-- CMakeLists.txt # cmake script to build shared lib
9  |-- src/
10 |-- main.c
```



CMake Configuration Files

./CMakeLists.txt:

```
1 cmake_minimum_required(VERSION 3.22.1)
2 project(makeexecutable)
3 set(CMAKE_BUILD_TYPE Release)
4 add_subdirectory(shared)
5 file(GLOB SOURCES "src/*.c")
6 add_executable(main ${SOURCES})
7 target_link_libraries(main mathutils)
```

shared/CMakeLists.txt:

```
1 add_library(mathutils SHARED mathutils.c)
2 target_include_directories(mathutils PUBLIC ${CMAKE_SOURCE_DIR}/include)
```



Build Commands and Resulting Structure

Build commands:

```
1 $ cd build
2 $ cmake ..
3 $ make
```

Updated Directory Tree:

```
1 .
2 |-- build/
3 |   |-- CMakeCache.txt
4 |   |-- CMakeFiles/
5 |   |-- |cmake_install.cmake|
6 |   |-- main
7 |   |-- Makefile
8 |   |-- shared/
9 |-- CMakeLists.txt
10 |-- include/
11 |   |-- mathutils.h
12 |-- shared/
13 |   |-- mathutils.c
14 |   |-- CMakeLists.txt
15 |-- src/
16 |   |-- main.c
```



Essential cmake Commands (Core vs Common)

Core (you will use constantly)

- **cmake_minimum_required(VERSION x.y)**
Set minimum CMake version + policies.
- **project(name [LANGUAGES])**
Define project + enabled languages.
- **add_executable(target srcs)**
Create an executable target.
- **add_library(target [STATIC|SHARED] srcs)**
Create a library target.
- **target_link_libraries(target libs)**
Link libraries to a target.
- **target_include_directories(target {PRIVATE|PUBLIC|INTERFACE} dirs)**
Add include paths with proper scope.
- **add_subdirectory(dir)**
Build subprojects / hierarchical layouts.

Common (very useful in real projects)

- **target_compile_options(target opts)**
Add compiler flags -Wall -Wextra -O2.
- **target_compile_definitions(target defs)**
Add -D macros (DEBUG, LOG_LEVEL).
- **find_package(Pkg ...)**
Locate external libraries (OpenSSL, Boost).
- **option(NAME "help" ON|OFF)**
Build-time switches (features, tests).
- **install(TARGETS ...)**
Installation rules.

Legacy / Use with care

- **file(GLOB var "pattern")**
May require running CMake when files change.



- **Official Documentation:**

- <https://cmake.org/documentation/>

- CMake Tutorial (Getting Started):

- [CMake Tutorial](#)

- A step-by-step walkthrough with code examples. These slides are based on the tutorial.

- Modern CMake Best Practices:

- [Modern CMake](#)

- Explains modern targets, interface libraries, and idioms (post v3.0).

- CMake GitHub Repository:

- github.com/Kitware/CMake

- Source code, issues, and development activity.

- **Books and Tutorials:**

- Professional CMake: A Practical Guide by Craig Scott



2. Linking



What is Linking?

Linking is the process of producing an executable by combining object files and resolving all symbol references across them.

Types of Linking

- **Static Linking:**

- All references to external symbols (from object files and libraries) are resolved at compile time.
- The resulting executable is self-contained.

- **Dynamic Linking:**

- References to other object files are resolved at compile time.
- References to shared libraries are resolved at load time or at first use (lazy binding).

Note: There are three actors at play here:

1. The static linker which resolves inter-module references and creates the executable in ELF format.
2. The loader which loads the executable and the dynamic linker
3. The dynamic linker that loads shared libraries and performs relocations.



Static Linking



Static Linking – essential steps

1. Input collection

- Read symbol table of each object file: what each file defines and references.

2. Symbol resolution

- For every undefined symbol, choose a matching definition. Report error for missing or multiply-defined symbols.

3. Combining layouts

- Merge like sections across modules: `.text`, `.rodata`, `.data`, `.bss`
- Assign final addresses, respecting alignment and memory permissions (R/X/W).

4. Relocation

- Patch code/data locations that reference symbols (calls, jumps, addresses).



The ELF File Format

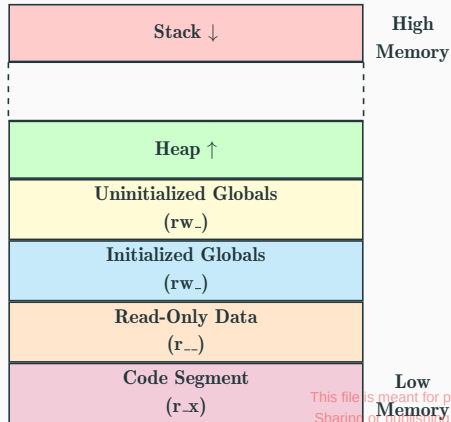
- In Linux, an ELF (Executable and Linkable Format) is a fixed structure for certain kinds of binary files.
- ELF files are used for:
 - Executables,
 - Object files (.o),
 - Shared libraries (.so), and
 - Core dumps.
- You can explore the structure of ELF files using tools like:
 - **readelf** — dump headers, symbols, relocations, sections
 - **objdump** — disassemble code, inspect sections
 - **nm** — list symbols
 - **file** — identify ELF file type



The Unix user-space memory model and the ELF format

The ELF format of the executable is structured to match the program's in-memory layout, so the kernel can map its sections directly into the right segments with the right permissions.

Memory segments



ELF sections

- .bss: for uninitialized global variables.
- .data: for initialized globals
- .rodata: for strings and other constants
- .text: for program instructions
- Symbol tables, relocation entries, headers.



An example

Module1.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  extern int b[];
5  int i;
6  const int *x = &(b[2]);
7  extern int fn();
8
9  int main() {
10         int* a = (int*) malloc(4);
11         i = fn();
12         printf("%d\n", b[0]);
13 }
```

Module2.c

```
1  int c = 58;
2  int d;
3  int b[] = {1, 2, 3};
4
5  int fn() {
6         static int b = 5;
7         int e = d = b;
8         return e;
9 }
```



Memory Segments for Module1.c and Module2.c

Module1.c

- Text segment: Contains the code for the function `main()`.
- Initialized data segment: Contains initialized global `x`.
- Uninitialized data segment: Contains uninitialized globals `i`.
- Heap: Memory allocated via `malloc(4)` in `main()`.
- Stack: Contains the local variable `a` in `main`.

Module2.c

- Text segment: Contains the code for the function `fn()`.
- Initialized data segment: Contains initialized globals: `int c = 58`, `int b[] = {1,2,3}`, and the static variable `static int b = 5` (inside `fn()`).
- Uninitialized data segment: Contains the uninitialized global variable `int d`.
- Heap: No dynamic memory allocations in this module.
- Stack: Contains `e` in `fn()`.



Tentative Symbol Addresses in object modules .o

- Each section (.text, .data, and .bss) is treated as relocatable and starts at address 0x0000000000000000.
- All declared entries (functions, variables, etc.) are given tentative addresses relative to their section's start.
- The actual absolute addresses are not determined until the linking stage.

Using the command `nm Module1.o`, you can view these symbols and their tentative addresses.

```
1 $ nm Module1.o
2 U b                # b is not defined in this module (U)
3 0000000000000000 B i  # address for i in .bss (B)
4 0000000000000000 T main # address for main in .text (T)
5 U malloc           # malloc is not defined in this module (U)
6 U printf           # printf is not defined in this module (U)
7 0000000000000000 D x  # address for x in .data (D)
```

A similar thing happens for `Module2.o`.



Combining ELF modules during linking

Module 1 sections

.bss i
.data x
.rodata "%d\n"
.text main

Module 2 sections

.bss d
.data c b[] b (static)
.rodata
.text fn

Combined sections after linking

.bss i d
.data x c b[] b (static)
.rodata "%d\n"
.text malloc printf fn main



Tracing ELF Symbols with nm

```
1 echo "Symbols in Module1.o"
2
3 # Module1.c symbols (example: main, x, i)
4 nm -n -S Module1.o | grep -E
5 '([^[:space:]](main|x|i))$'
6
7 echo "-----"
8
9 echo "Symbols in Module2.o"
10
11 # Module2.c symbols (example: fn, c, d, b)
12 nm -n -S Module2.o | grep -E
13 '([^[:space:]](fn|c|d|b|b.0))$'
14
15 echo "-----"
16
17 echo "Symbols in the final executable"
18
19 # Module2.c symbols (example: fn, c, d, b)
20 nm -n -S Module | grep -E
21 '([^[:space:]](main|x|i|fn|c|d|b|b.0))$'
```

```
1 Symbols in Module1.o
2 0000000000000000 0000000000000004 B i
3 0000000000000000 000000000000004d T main
4 0000000000000000 0000000000000008 D x
5 -----
6 Symbols in Module2.o
7 0000000000000000 0000000000000004 D c
8 0000000000000000 0000000000000004 B d
9 0000000000000000 0000000000000022 T fn
10 0000000000000008 000000000000000c D b
11 0000000000000014 0000000000000004 d b.0
12 -----
13 Symbols in the final executable
14 0000000000401745 000000000000004d T main
15 0000000000401792 0000000000000022 T fn
16 00000000004c50f0 0000000000000008 D x
17 00000000004c50f8 0000000000000004 D c
18 00000000004c5100 000000000000000c D b
19 00000000004c510c 0000000000000004 d b.0
20 00000000004c7310 0000000000000004 B i
21 00000000004c7314 0000000000000004 B d
```



Issues in relocation

- In the object `.o` files, the compiler emits code to **refer** to functions and variables.
 - What do these references look like in the machine code?
- During linking, object files are **combined** and like-sections are **merged**, moving the functions and variables to new locations.
- Relocation **updates** every affected reference so that each `call/jump/load/store` points to the symbol's new final address.
 - Which references are affected and need relocation? And how are they updated?



So how are symbols referred to in object files?

To see this use the command: `objdump -dr Module1.o;`

`-d`: disassemble machine code,

`-r`: show relocation entries

```
1 ; assembly for i = fn()
2 1f: e8 00 00 00 00      call    24 <main+0x24>
3 20: R_X86_64_PLT32 fn-0x4
4
5 24: 89 05 00 00 00 00    mov     %eax,0x0(%rip)      # 2a <main+0x2a>
6 26: R_X86_64_PC32 i-0x4
7 ...
```

- **call: e8** = relative call; the 4-byte displacement is currently 20: 00 00 00 00, a placeholder. The linker will patch this call to get the address of `fn`.
- **mov %eax, 0x0(%rip)** stores the return value in `%eax` into memory. The linker will patch the placeholder displacement 26: 00 00 00 00 to reach the address of `i`.
- **These zero displacements are only temporary: The linker does relocation to fix these later.**



What is PC relative addressing?

What is PC-relative addressing?

```
1 $ objdump -d main | grep "<fn>"
2 401764:      e8 29 00 00 00          call    401792 <fn>
3 401769:
4 0000000000401792 <fn>:
```

- Opcode: **e8** = near call
- Operand: **29 00 00 00** (little-endian) = **0x29** (the displacement)
- Instruction address: **0x401764**; PC after call = **0x401769**
- Target = **0x401769 + 0x29 = 0x401792** = address of **f**
- If caller and the callee are in the same section, they move together if relocated, and no relocation is needed.



Relocation of Symbol References

- **Case 1:** The reference is a call to a function in the same module
 - A PC-relative call instruction with an displacement of x is generated.
 - Since both are in the same code segment, this displacement remains valid even after relocation.
 - **No relocation is required during linking.**
- **Case 2:** Call to a function in a different module
 - The displacement is not known initially.
 - The compiler generates a relocation entry.
 - The linker fills in the correct displacement during linking.
 - **Relocation is required.**

Case 3: Reference to a variable in the same module

- Code and data reside in different sections.
- This case is similar to Case 2 for static linking.
- **Case 4:** Reference to a variable in another module
 - Similar to Case 2 for static linking.



Relocation Records

- The linker uses the Relocation Table to adjust the addresses of symbols in final executable. These are generated by the compiler and stored in the object files.

```
1 $readelf -r Module1.o
2 Module1.o:      file format elf64-x86-64
3
4 RELOCATION RECORDS FOR [.text]:
5
6 OFFSET          TYPE             VALUE
7 00000000000012   R_X86_64_PLT32 0000000000000000 malloc-4
8 00000000000020   R_X86_64_PLT32 0000000000000000 fn - 4
9 00000000000026   R_X86_64_PC32  0000000000000000 i - 4
10 0000000000002c   R_X86_64_PC32  0000000000000000 b - 4
11 00000000000035   R_X86_64_PC32  0000000000000000 .rodata
    - 4
12 00000000000042   R_X86_64_PLT32 0000000000000000 printf -
    4
```

```
1 OFFSET - Address where the displacement is
        written
2 addend - The constant addend (often -4 for
        call/jmp rel32)
3 target - Final address of the referenced
        symbol/PLT entry
4 ---
5 Rule (x86-64 rel32):
6 OFFSET + 4 + displacement = target
7 displacement = target - (OFFSET + 4)
8
9 In readelf's notation, OFFSET is implicit.
10 displacement = target - addend
11 Example: malloc - 4
```

Question: What displacement should the linker enter at 00000000000012 during relocation?

Answer: “Take the final address of malloc, subtract 4 and then subtract 00000000000012”.



Dynamic Linking



Dynamic linking benefits

Dynamic linking is the process of resolving library references at load time or runtime, rather than embedding the code into the executable at compile time.

- **Smaller Executable Size:** Executables don't need to include the library code, resulting in smaller file sizes.
- **Reduced Memory Footprint:** The libraries created for dynamic linking (also called shared libraries) are loaded into memory once and shared among multiple processes, reducing overall physical memory usage.
- **Easier Updates and Maintenance:** When a shared library is updated, programs that use it need not be recompiled.



How is the linker of GCC (g++/collect2) invoked?

```
1 /usr/lib/gcc/x86_64-linux-gnu/11/collect2 \           // Linker driver of GCC, the linker is ld
2 -plugin /usr/lib/gcc/x86_64-linux-gnu/11/liblto_plugin.so \
3 -plugin-opt=/usr/lib/gcc/x86_64-linux-gnu/11/lto-wrapper \
4 -plugin-opt=-fresolution=example1.res \
5 ...
6 -dynamic-linker /lib64/ld-linux-x86-64.so.2 \       // Set the dynamic linker.
7 -pie \
8 -z now -z relro \
9 -o example1 \                                       // Output executable name
10 -L/usr/lib/gcc/x86_64-linux-gnu/11                // Library paths
11 -L/lib/x86_64-linux-gnu
12 -L/usr/lib/x86_64-linux-gnu
13 ... \
14 example1.o \   // Your object file(s)
15 -lstdc++ -lm -lgcc_s -lgcc -lc \                 // C++ stdlib, math, GCC runtime/unwind, and C library
16 (glibc)
17 ... \
18 /usr/lib/.../crtendS.o \
19 /usr/lib/.../crti.o
```



Dynamic Linking: Build time vs Runtime phases

In dynamic linking, there are two distinct phases:

1. Build time (static linking) phase:

- Interdependences between user modules (e.g., `Module1.o` and `Module2.o`) are fully resolved.
- However, dependencies on (shared) libraries remain unresolved during this phase

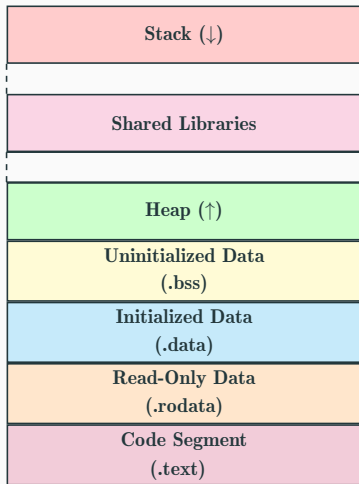
2. Runtime (dynamic Linking) phase:

- `execve` starts execution of the ELF executable.
- The kernel reads the ELF program headers and maps the executable into the process address space (commonly called loading).
- If a `PT_INTERP` entry is present, the kernel also maps the dynamic linker into the process address space and transfers control to it (do a `readelf -p .interp` to see the dynamic linker path).
- The dynamic linker maps (loads) the required shared libraries, performs the necessary relocations, and prepares the program for execution.
- Shared-library variables are usually resolved during this startup phase, while shared-library functions are often resolved lazily on first call through the PLT/GOT mechanism.

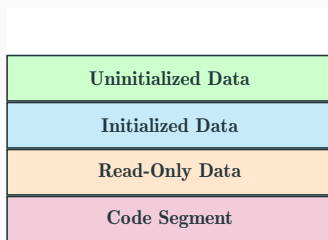


User space memory segments for dynamic linking

High Memory



Expansion of Shared Library (e.g., libc.so)



- The `.text` segment of a library may be shared by other libraries and modules.
- A shared library, however, does not have a separate stack. Its stack grows on the common stack.
- Likewise, a shared library's heap grows on the common heap.



Dynamic Linking: The whole story

Dynamic linking involves multiple stages that occur across build-time and runtime:

1. Pre-linking Phase:

- Shared libraries are compiled using `-fPIC`, enabling Position-Independent Code.
- The executable is compiled using `-L<path> -l<libname>` to specify the path and name of required libraries.
- No actual resolution happens yet, but references to external symbols are recorded.

2. Static Linking Phase:

- All references between user object files (e.g., `Module1.o`, `Module2.o`) are fully resolved.
- The linker verifies that the required shared libraries (e.g., `libm.so`) contain the referenced symbols.
- Relocation records and stubs (PLT/GOT entries) are created for external symbols.



Dynamic Linking: The whole story (continued)

Once the executable is launched:

3. Load-Time Linking

- The dynamic linker (e.g., `ld-linux.so`) is invoked by the kernel.
- Shared libraries are mapped into memory (usually between heap and stack). These libraries are shared amongst processes.
- Some symbols (e.g., global vars) are resolved immediately, and the GOT is updated.
- However, function calls are not resolved till their first use.

4. Lazy Binding (elaborated in the following slides)

- Function calls are redirected via the PLT.
- The first call triggers a jump through GOT to `PLT[0]`, which invokes the dynamic linker.
- The dynamic linker finds the symbol, patches the GOT, and the call proceeds.
- On subsequent calls, the updated GOT provides a direct jump to the function.



The PLT/GOT jugalbandi

C Program

```
1  #include <stdio.h>
2  int main() {
3      printf("Hi\n");
4      return 0;
5  }
```

Assembly for printf in executable

```
1  main:
2      ...
3      callq PLT[printf]
4      ...
```

PLT Table (labeled code fragments)

```
1  PLT[0]:
2  pushq *GOT[1]; Address of link map.
3  jump  *GOT[2]; jump to resolver of dynamic linker
4  ...
5  PLT[printf]:
6  jmpq  *GOT[printf]
7  next: pushq $index; relocation index for printf
8  jmpq  PLT[0]
```

GOT Table (in data segment)

```
1  GOT[printf] (initially): label next: in PLT table
2  GOT[printf] (after resolution): address of printf in
   libc.so
3  GOT[1]: → address of link map
4  GOT[2]: → address of resolver of dynamic linker
```




Lazy Binding via PLT/GOT: `printf()` Example

1. A call to `printf()` transfers control to `PLT[printf]`.
2. The PLT is a table of code fragments. The GOT is a table of data.
3. The code at `PLT[printf]` jumps to the address in `GOT[printf]`. Initially, this address points back into `PLT[printf]` (the instruction labeled `next:`).
4. A relocation index `$index` is pushed onto the stack. Control is transferred to `PLT[0]`.
5. `PLT[0]` pushes the address of the link map (i.e., `GOT[1]`) onto the stack.
6. Control jumps to the dynamic linker resolver (address stored in `GOT[2]`).
7. Using the link map and the `$index`, the resolver identifies the symbol being called — here, `printf` — and finds its address in `libc.so`.
8. `GOT[printf]` is updated with the resolved runtime address of `printf`. Control is transferred to that address.
9. Any subsequent calls to `printf()` now directly jump to the correct address via `GOT[printf]` through `PLT[printf]`.



Binary Creation: Summary

- Binary creation is the gradual transformation of a program from human-readable source into an executable form.
 - Along the way, the program passes through several representations.
 - The compilation pipeline is a progression from description to realization.
- For large scale projects, the binary creation process is aided by build tools such as `make` and `CMake`.
 - These tools capture dependencies, track updates, and ensure efficiency through incremental compilation.
- Linking is the resolution of external references.
 - Static linking produces a binary whose external requirements (from libraries) has already been absorbed into the executable.
 - Dynamic linking postpones part of the completion until runtime, when shared libraries are mapped, symbols are resolved, and execution is finalized.
- In this sense, binary creation begins with translation, is managed by build structure, and may continue all the way into execution.



Recommended Resources on Linking and Loading

- **Computer Systems: A Programmer's Perspective (aka CSAPP); Randal E. Bryant and David R. O'Hallaron, Pearson Education India; 3rd edition, 2016;**
 - Chapter 7 (Linking) offers a superb explanation of static and dynamic linking.
 - Clear discussion of ELF format, PLT, GOT, and lazy binding.
- **Linkers and Loaders; John R. Levine, Morgan Kaufmann; 1999**
 - Classic book focused exclusively on linking, loading, symbol resolution, relocation.
- **The ELF Specification**
 - Official format reference: <https://refspecs.linuxfoundation.org/elf/>. Our interest is in the **x86-64 abi**
- **man pages and tooling guides**
 - `man ld`, `man objdump`, `man readelf`, `man nm`
- **Articles and Tutorials**
 - "How Programs Get Run" by Ulrich Drepper (could not find this, let me know if you do)
 - "Understanding ELF Using readelf and objdump" — LWN.net